

3.

## Exceptions and Error handling



ZeroDivisionError is raised when a program attempts to perform a division operation where the denominator is zero.

Syntax: `ZeroDivisionError: division by zero`

Why :

**Direct Division by Zero**

Code:  3\_01\_exceptionHandling.py > ...

```
1  numerator = 10
2  denominator = 0
3  result = numerator / denominator
4  print(result)
```

**Why : Variable Initialization Issue****Code:** 3\_01\_exceptionHandling.py > [🔗] denominator

```
1 denominator = 0
```

```
2 result = 20 / denominator
```

**Why : Conditional Statements Issue****Code:** 3\_01\_exceptionHandling.py > ...

```
1 denominator = 0
```

```
2 if denominator != 0:
```

```
3     result = 25 / denominator
```

Approach/Reason to Solve Zerodivisionerror

Use if-else

Code:  3\_01\_exceptionHandlingSol.py > ...

```
1   numerator = 10
2   denominator = 0
3
4   if denominator != 0:
5       |   result = numerator / denominator
6   else:
7       |   print("Error: Cannot divide by zero")
```

## Using Conditional Expression

1 3\_01\_exceptionHandlingSol.py > ...

2 1 numerator = 10

3 2 denominator = 0

4 3

4 4 result = numerator / denominator if denominator != 0 else "Error: Denominator cannot be zero"

5 5 print(result)

## Catching the Exception (Using Try-Except Block)

8 3\_01\_exceptionHandlingSol.py > ...

9 1 numerator = 10

10 2 denominator = 0

11 3 try:

12 4 | result = numerator / denominator

13 5 except ZeroDivisionError:

14 6 | print("Error: Cannot divide by zero")

Input a valid number

3\_01\_exceptionHandlingSolExample.py > ...

```
1 while True:
2     try:
3         x = int(input("Please enter a number: "))
4         break
5     except ValueError:
6         print("Oops! That was no valid number. Try again...")
7 print(x)
```

Sometimes it is necessary to raise an exception deliberately if a condition occurs that doesn't allow your program to proceed. You can raise exceptions using the statement.

```
course6 > 1_03_RaisingAnException.py > ...
1  def check_age(age):
2      if age < 0:
3          raise ValueError("Age cannot be negative.")
4      elif age < 18:
5          print("You are not old enough.")
6      else:
7          print("You are welcome.")
8  try:
9      user_age = int(input("Enter your age: "))
10     check_age(user_age)
11 except ValueError as e:
12     print("Error:", e)
```

By raising an exception, you can enforce certain conditions within your programs. It's particularly useful in data validation, where you need to ensure that input data conforms to expected parameters.

1 You can also define your own exceptions by extending the class. This  
2 is useful when you need to create custom error messages for specific  
3 business logic in your application. (Class, OOP)  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14



Code:

```
3_02_errorHandling.py > ...  
1  # bad practice  
2  def read_file(file_path):  
3      file = open(file_path, 'r')  
4      content = file.read()  
5      file.close()  
6      return content  
7  print(read_file('textError.txt'))  
8  
9  # good practice  
10 def read_file(file_path):  
11     try:  
12         with open(file_path, 'r') as file:  
13             return file.read()  
14     except FileNotFoundError:  
15         return "File not found."  
16 print(read_file('text_Error.txt'))
```